



Cyberscope

Audit Report

Bitnium

May 2026

Network ETH

Address 0x468156a1187cf4cd8f009a59772f96a5287d1cc0

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	IDI	Immutable Declaration Improvement	Unresolved
●	IFCC	Incorrect Fee Cap Check	Unresolved
●	MFCC	Missing Fee Cap Check	Unresolved
●	PAC	Partial Allowance Check	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L16	Validate Variable Setters	Unresolved

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	3
Risk Classification	4
Review	5
Audit Updates	5
Source Files	5
Findings Breakdown	6
IDI - Immutable Declaration Improvement	7
Description	7
Recommendation	7
IFCC - Incorrect Fee Cap Check	8
Description	8
Recommendation	8
MFCC - Missing Fee Cap Check	9
Description	9
Recommendation	9
PAC - Partial Allowance Check	10
Description	10
Recommendation	10
L04 - Conformance to Solidity Naming Conventions	11
Description	11
Recommendation	11
L16 - Validate Variable Setters	13
Description	13
Recommendation	13
Functions Analysis	14
Inheritance Graph	17
Flow Graph	18
Summary	19
Disclaimer	20
About Cyberscope	21

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	Bitnium
Explorer	https://etherscan.io/address/0x468156a1187cf4cd8f009a59772f96a5287d1cc0
Symbol	BTNM
Decimals	18
Total Supply	21,000,000

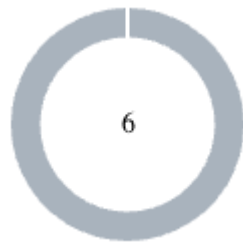
Audit Updates

Initial Audit	04 May 2026
---------------	-------------

Source Files

Filename	SHA256
contracts/ReflectiveERC20.sol	8e9820a2678789110e1fbad978c1c86ae642cc129f2b9bcea6766792ccdd02fd
contracts/Bitnium.sol	f1f302b68706a28fcf8de291a56db143ecbe166c9ada7c4a84c578289ac9e1be
contracts/lib/LibCommon.sol	ad40e79524942f0927be19739e7c96b7a52147f5cf54fdd7eb676720db70b66a

Findings Breakdown



● Critical	0
● Medium	0
● Minor / Informative	6

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	0	0	0
● Minor / Informative	6	0	0	0

IDI - Immutable Declaration Improvement

Criticality	Minor / Informative
Location	contracts/Bitnium.sol#L150
Status	Unresolved

Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variables that saves gas when it is defined.

```
maxTotalSupply
```

Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.

IFCC - Incorrect Fee Cap Check

Criticality	Minor / Informative
Location	Bitnium.sol#L300,L304
Status	Unresolved

Description

`setDeflationConfig` validates the updated aggregate fee using the current `deflationBPS` instead of the current `taxBPS`. This causes the check to add the old deflation fee and the new deflation fee together, while excluding the tax fee from the total.

As a result, the function may incorrectly allow configurations where `taxBPS + tFeeBPS + _deflationBPS` exceeds `MAX_ALLOWED_BPS`, or incorrectly reject valid configurations when the previous deflation fee is high.

```
function setDeflationConfig(uint256 _deflationBPS) external onlyOwner {
    if (!isDeflationary()) {
        revert TokenIsNotDeflationary();
    }
    uint256 totalBPS = deflationBPS + tFeeBPS + _deflationBPS;
    if (totalBPS > MAX_ALLOWED_BPS) {
        revert InvalidTotalBPS(totalBPS);
    }
    deflationBPS = _deflationBPS;
    emit DeflationConfigSet(_deflationBPS);
}
```

Recommendation

Update the deflation fee validation to calculate the aggregate fee from the current tax fee, current reflection fee, and proposed deflation fee. Ensure all fee update paths consistently enforce the same maximum total fee invariant.

MFCC - Missing Fee Cap Check

Criticality	Minor / Informative
Location	Bitnium.sol#L157,L174,L266
Status	Unresolved

Description

`setReflectionConfig` updates the reflection fee without enforcing the same aggregate fee ceiling applied during initialization. The constructor validates that enabled tax, deflation, and reflection fees do not exceed `MAX_ALLOWED_BPS`, but the reflection setter only checks that reflection is enabled before updating `tFeeBPS`.

As a result, the owner can later increase `_feeBPS` so that `taxBPS + deflationBPS + _feeBPS` exceeds the intended maximum fee limit, bypassing the constructor's invariant.

```
if (totalBPS > MAX_ALLOWED_BPS) {
    revert InvalidTotalBPS(totalBPS);
}

function setReflectionConfig(uint256 _feeBPS) external onlyOwner {
    if (!isReflective()) {
        revert TokenIsNotReflective();
    }
    super._setReflectionFee(_feeBPS);

    emit ReflectionConfigSet(_feeBPS);
}
```

Recommendation

Apply the same aggregate basis-points validation in every fee update path. Before updating the reflection fee, calculate the total configured fee using the current tax and deflation values plus the new reflection fee, and revert if it exceeds the maximum allowed threshold.

PAC - Partial Allowance Check

Criticality	Minor / Informative
Location	Bitnium.sol#L346,L368
Status	Unresolved

Description

`Bitnium.transferFrom` calculates tax and deflation amounts from the requested `amount`, applies those charges to `from`, and then calls `super.transferFrom` only with `amountToTransfer`. The parent implementation spends allowance for the value it receives, so allowance is checked and reduced only for the net recipient amount rather than the full amount deducted from the sender.

This lets a spender transfer an amount larger than their approved allowance when fees are enabled, because the tax and deflation portions are taken from `from` without being included in the allowance check.

```
uint256 taxAmount = _taxAmount(from, amount);
uint256 deflationAmount = _deflationAmount(amount);
uint256 amountToTransfer = amount - taxAmount - deflationAmount;

if (taxAmount != 0) {
    _transferNonReflectedTax(from, taxAddress, taxAmount);
}
if (deflationAmount != 0) {
    _burn(from, deflationAmount);
}

return super.transferFrom(from, to, amountToTransfer);
```

Recommendation

Ensure delegated transfers validate and consume allowance for the full amount charged to the token owner. The allowance accounting should cover the recipient transfer, tax transfer, and burn amount consistently before any balance-changing operations are performed.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	contracts/ReflectiveERC20.sol#L38,221 contracts/Bitnium.sol#L160,266,280,281,300
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
function _tTotal() public view virtual returns (uint256) {  
    return totalSupply();  
}  
  
uint256 _amount  
address _taxAddress  
uint256 _feeBPS  
uint256 _taxBPS  
uint256 _deflationBPS
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	contracts/Bitnium.sol#L138,145,292
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
taxAddress = _taxAddress  
initialTokenOwner = tokenOwner
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

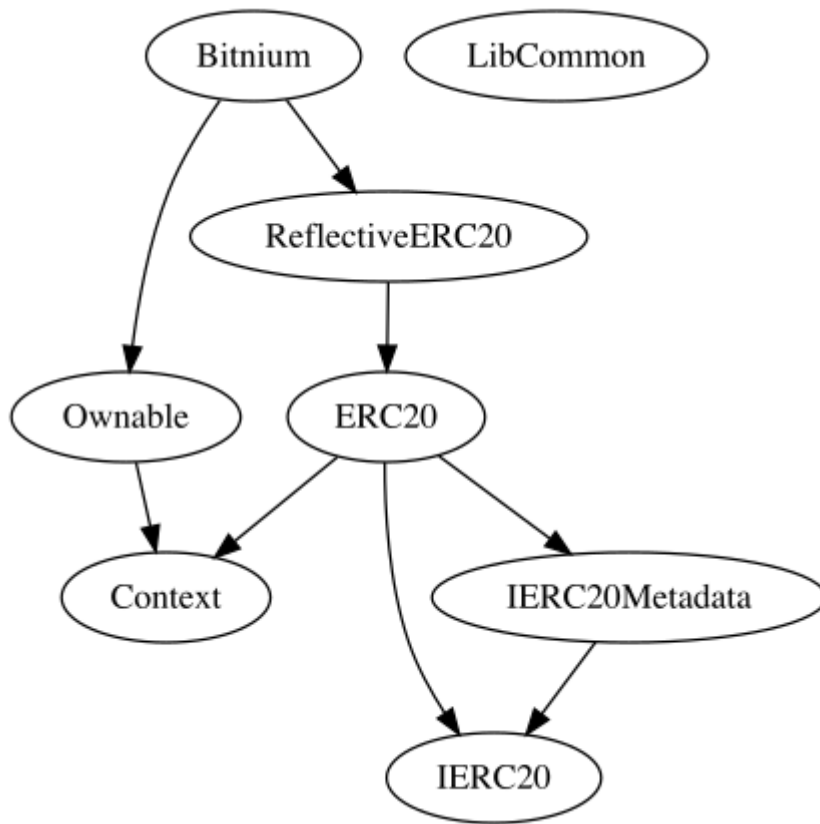
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
ReflectiveERC20	Implementation	ERC20		
	_tTotal	Public		-
		Public	✓	ERC20
	balanceOf	Public		-
	transferFrom	Public	✓	-
	transfer	Public	✓	-
	_transfer	Internal	✓	
	_mint	Internal	✓	
	_burn	Internal	✓	
	_setReflectionFee	Internal	✓	
	tokenFromReflection	Public		-
	_transferReflected	Private	✓	
	_reflectFee	Private	✓	
	calculateFee	Private		
	_transferNonReflectedTax	Internal	✓	
	_getRValues	Private		
	_getRate	Private		
	_getCurrentSupply	Private		
	_rUpdate	Private	✓	

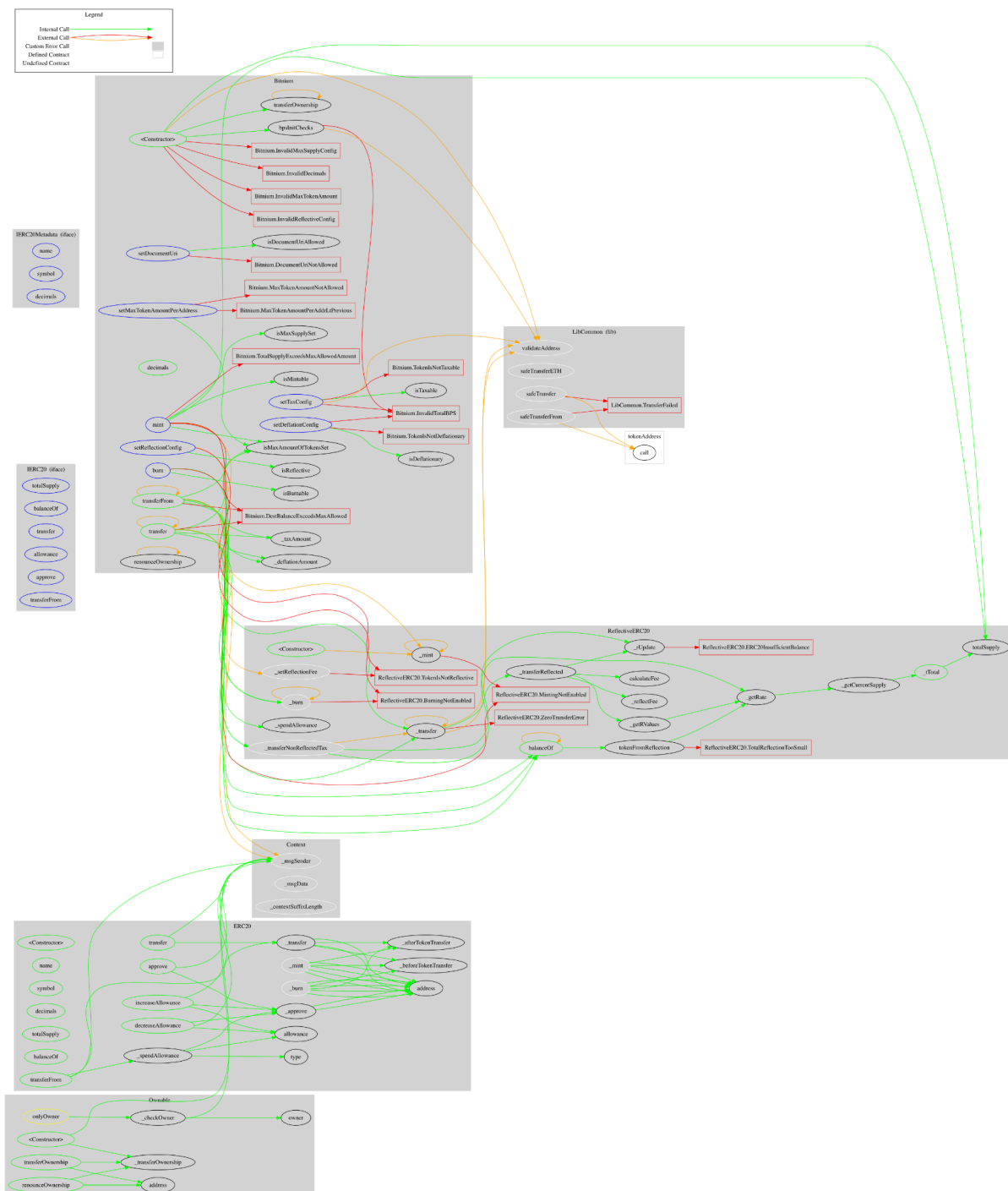
Bitnium	Implementation	ReflectiveERC20, Ownable		
		Public	✓	ReflectiveERC20
	bpsInitChecks	Private		
	isMintable	Public		-
	isBurnable	Public		-
	isMaxAmountOfTokensSet	Public		-
	isMaxSupplySet	Public		-
	isDocumentUriAllowed	Public		-
	decimals	Public		-
	isTaxable	Public		-
	isDeflationary	Public		-
	isReflective	Public		-
	setDocumentUri	External	✓	onlyOwner
	setMaxTokenAmountPerAddress	External	✓	onlyOwner
	setReflectionConfig	External	✓	onlyOwner
	setTaxConfig	External	✓	onlyOwner
	setDeflationConfig	External	✓	onlyOwner
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	mint	External	✓	onlyOwner
	burn	External	✓	onlyOwner
	renounceOwnership	Public	✓	onlyOwner

	transferOwnership	Public	✓	onlyOwner
	_taxAmount	Internal		
	_deflationAmount	Internal		

Inheritance Graph



Flow Graph



Summary

Bitnium contract implements a token mechanism. This audit investigates security issues, business logic concerns, and potential improvements. Bitnium is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler errors or critical issues. The contract Owner can access some admin functions that can not be used in a malicious way to disturb the users' transactions.

The contract's ownership has been renounced. The information regarding the transaction can be accessed through the following link:

<https://etherscan.io/tx/0xa1abc95f4ecc64b33a14b1693f71bf766e8aec9aa041790c2e12817be8a252a7>

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



The Cyberscope team

cyberscope.io